

Title: E-Commerce CRUD Project with Django

INTRODUCTION

This project demonstrates how to build a complete mini e-commerce application using Django 5, Python 3.12+, MySQL database, and GitHub for version control. The application implements essential features including user authentication, role-based access control (RBAC), CRUD operations for products and categories, a persistent shopping cart system, order management, and sales reporting.

The main objectives of this project are:

1. Understanding database connectivity and configuration
2. Implementing authentication and authorization systems
3. Performing CRUD operations with Django ORM
4. Managing file uploads (product images)
5. Using Git and GitHub for version control
6. Building role-based middleware for access control

STEP 1: ENVIRONMENT SETUP

1.1 Installing Python 3.12+

Python is a high-level, interpreted programming language widely used for web development. Django is a Python web framework that follows the Model-View-Template (MVT) architectural pattern.

I downloaded Python 3.12 from python.org and verified the installation: python

```
--version
```

Output: Python 3.12.x

1.2 Installing MySQL Database

MySQL is a relational database management system. I installed MySQL Community Server and set up the root password during installation.

```
mysql --version
```

Output: mysql Ver 8.0.x

1.3 Creating Virtual Environment

A virtual environment isolates project dependencies from the global Python installation.

Commands **Used:** mkdir

ecommerce_lab cd

ecommerce_lab python -m venv

venv venv\Scripts\activate #

Windows

After activation, the terminal shows (venv) indicating the virtual environment is active.

1.4 Installing Django and Dependencies

pip install django==4.2 pip install

mysqlclient pip install pillow

- **Django 4.2:** Web framework (LTS version compatible with MariaDB 10.4+)
- **mysqlclient:** Python MySQL database connector
- **Pillow:** Image processing library for file uploads

1.5 Creating Django Project and App

django-admin startproject ecommerce .

python manage.py startapp shop

This creates:

- **ecommerce/** - Project configuration folder
- **shop/** - Main application folder
- **manage.py** - Django management script

STEP 2: DATABASE CONFIGURATION

2.1 Creating MySQL Database

I logged into MySQL and created a database for the project: mysql

-u root -p

CREATE DATABASE ecommerce_lab3;

CREATE USER 'django_user'@'localhost' IDENTIFIED BY 'django_password'; GRANT ALL PRIVILEGES ON ecommerce_lab3.* TO 'django_user'@'localhost';

```
FLUSH PRIVILEGES;
```

```
EXIT;
```

2.2 Configuring Django Settings

In ecommerce/settings.py, I configured the database connection and added the shop app:

```
INSTALLED_APPS = [
```

```
    'django.contrib.admin',
```

```
    'django.contrib.auth',
```

```
    'django.contrib.contenttypes',
```

```
    'django.contrib.sessions',
```

```
    'django.contrib.messages',
```

```
    'django.contrib.staticfiles',
```

```
    'shop', # Added
```

```
]
```

```
DATABASES = {
```

```
    'default': {
```

```
        'ENGINE': 'django.db.backends.mysql',
```

```
        'NAME': 'ecommerce_lab3',
```

```
        'USER': 'django_user',
```

```
        'PASSWORD': 'django_password',
```

```
        'HOST': 'localhost',
```

```
        'PORT': '3306',
```

```
    }
```

```
}
```

```
# Template configuration
```

```
TEMPLATES = [
```

```
    {
```

```
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
```

```

'DIRS': [BASE_DIR / 'templates'],
'APP_DIRS': True,
# ... rest of config
},
]

```

Media files for image uploads

```
MEDIA_URL = '/media/'
```

```
MEDIA_ROOT = BASE_DIR / 'media'
```

2.3 Running Initial Migrations python

```
manage.py migrate
```

This creates default Django tables in the MySQL database (auth_user, sessions, etc.).

STEP 3: CREATING MODELS

Models define the database schema using Django's ORM (Object-Relational Mapping).

3.1 Database Models (shop/models.py)

```
from django.db import models from
```

```
django.contrib.auth.models import User
```

```
class Role(models.Model):
```

```
    ROLE_CHOICES = [
```

```
        ('viewer', 'Viewer'),
```

```
        ('accountant', 'Accountant'),
```

```
        ('admin', 'Admin'),
```

```
    ]
```

```
    user = models.OneToOneField(User, on_delete=models.CASCADE)    role =
models.CharField(max_length=20, choices=ROLE_CHOICES, default='viewer')    def
__str__(self):
```

```
        return f"{self.user.username} - {self.role}"
```

```
class Category(models.Model):  
    name = models.CharField(max_length=100)  
    description = models.TextField(blank=True)    created_at =  
    models.DateTimeField(auto_now_add=True)
```

```
    def __str__(self):  
    return self.name
```

```
class Product(models.Model):  
    name = models.CharField(max_length=200)    description =  
    models.TextField()    price = models.DecimalField(max_digits=10,  
    decimal_places=2)    stock = models.IntegerField(default=0)    category =  
    models.ForeignKey(Category, on_delete=models.CASCADE)    image =  
    models.ImageField(upload_to='products/', blank=True, null=True)  
    created_at = models.DateTimeField(auto_now_add=True)
```

```
    def __str__(self):  
    return self.name
```

```
class Cart(models.Model):  
    user = models.ForeignKey(User, on_delete=models.CASCADE)  
    product = models.ForeignKey(Product, on_delete=models.CASCADE)  
    quantity = models.IntegerField(default=1)  
  
    added_at = models.DateTimeField(auto_now_add=True)
```

```
    def get_total(self):
```

```
return self.product.price * self.quantity
```

```
class Order(models.Model):
```

```
    user = models.ForeignKey(User, on_delete=models.CASCADE)
```

```
    total_amount = models.DecimalField(max_digits=10, decimal_places=2)
```

```
    created_at = models.DateTimeField(auto_now_add=True)    status =
```

```
    models.CharField(max_length=20, default='pending')
```

```
class OrderItem(models.Model):
```

```
    order = models.ForeignKey(Order, on_delete=models.CASCADE,
```

```
    related_name='items')    product = models.ForeignKey(Product,
```

```
    on_delete=models.CASCADE)    quantity = models.IntegerField()    price =
```

```
    models.DecimalField(max_digits=10, decimal_places=2)
```

3.2 Creating and Applying Migrations

```
python manage.py makemigrations python
```

```
manage.py migrate
```

This creates tables: shop_role, shop_category, shop_product, shop_cart, shop_order, shop_orderitem.

STEP 4: ROLE-BASED ACCESS CONTROL

4.1 Creating Role Decorator (shop/decorators.py)

```
from django.shortcuts import redirect from
```

```
django.contrib import messages from functools
```

```
import wraps
```

```
def role_required(allowed_roles=[]):    def
```

```
decorator(view_func):
```

```
    @wraps(view_func)    def
```

```

wrapper(request, *args, **kwargs):
    if not request.user.is_authenticated:
        return redirect('login')

    try:
        user_role = request.user.role.role
        if user_role in
allowed_roles or request.user.is_superuser:
            return view_func(request, *args, **kwargs)
        else:
            messages.error(request, 'Access denied.')
    except:
        messages.error(request, 'Role not assigned.')
    return redirect('home')
return wrapper
decorator

```

This decorator restricts access based on user roles:

- **Viewer:** Can browse products and shop
- **Accountant:** Can view orders and sales reports
- **Admin:** Full CRUD access to products and categories

STEP 5: AUTHENTICATION SYSTEM

5.1 Authentication Views (shop/views.py)

```

from
django.shortcuts import render, redirect
from
django.contrib.auth import login, authenticate, logout
from
django.contrib.auth.models import User
from
django.contrib import messages
from .models import Role

```

```

def signup_view(request):
    if request.method == 'POST':
        username = request.POST['username']
        email = request.POST['email']
        password = request.POST['password']
        password2 = request.POST['password2']

        if password == password2:
            if User.objects.filter(username=username).exists():
                messages.error(request, 'Username already exists')
            else:
                user = User.objects.create_user(username=username, email=email,
                password=password)
                Role.objects.create(user=user, role='viewer')
                messages.success(request, 'Account created!')
            return redirect('login')
        else:
            messages.error(request, 'Passwords do not match')
        return render(request, 'signup.html')

def login_view(request):
    if request.method == 'POST':
        username = request.POST['username']
        password = request.POST['password']
        user = authenticate(request, username=username, password=password)
        if user:
            login(request, user)
        return redirect('home')
    else:

```



```
        messages.error(request, 'Invalid credentials')
return render(request, 'login.html')
```

```
def logout_view(request):
    logout(request)    return
    redirect('home')
```

All new signups automatically receive the "Viewer" role. Roles can be upgraded via Django admin panel.

STEP 6: CRUD OPERATIONS

6.1 Category CRUD Views

```
@role_required(['admin']) def
category_list(request):
    categories = Category.objects.all()    return render(request,
'admin/category_list.html', {'categories': categories})
```

```
@role_required(['admin']) def
category_create(request):    if request.method
== 'POST':        name =
request.POST['name']        description =
request.POST['description']
        Category.objects.create(name=name, description=description) messages.success(request,
'Category created!') return redirect('category_list')    return render(request,
'admin/category_form.html')
```

```
@role_required(['admin']) def
category_update(request, pk):
```

```

        category = get_object_or_404(Category, pk=pk)
    if request.method == 'POST':
        category.name = request.POST['name']      category.description =
request.POST['description']      category.save()
    messages.success(request, 'Category updated!')      return
    redirect('category_list')      return render(request,
'admin/category_form.html', {'category': category})

```

```

@role_required(['admin']) def
category_delete(request, pk):
    category = get_object_or_404(Category,
pk=pk)      category.delete()
    messages.success(request, 'Category deleted!')
    return redirect('category_list')

```

6.2 Product CRUD Views

Similar CRUD operations were implemented for products with image upload functionality:

```

@role_required(['admin']) def
product_create(request):      if
request.method == 'POST':
    name = request.POST['name']
    description =
request.POST['description'] price
= request.POST['price']      stock
= request.POST['stock']
    category_id =
request.POST['category']

```

```

image =
request.FILES.get('image')

        Product.objects.create(
name=name,
description=description,
price=price,          stock=stock,
category_id=category_id,
image=image
        )
        messages.success(request, 'Product created!')
return redirect('product_list')

categories = Category.objects.all()    return render(request,
'admin/product_form.html', {'categories': categories})

```

STEP 7: E-COMMERCE FEATURES

7.1 Public Shopping Pages Home

Page displays featured products: def

```

home_view(request):
    products = Product.objects.all()[:8]    return
render(request, 'home.html', {'products': products})

```

Shop Page with category filtering: def

```

shop_view(request):    category_id =
request.GET.get('category')    if
category_id:
    products = Product.objects.filter(category_id=category_id)
else:

```

```
        products = Product.objects.all()    categories = Category.objects.all()    return  
render(request, 'shop.html', {'products': products, 'categories': categories})
```

Product Detail Page:

```
def product_detail_view(request, pk):  
    product = get_object_or_404(Product, pk=pk)    return  
render(request, 'product_detail.html', {'product': product})
```

7.2 Shopping Cart System

The cart is persistent (stored in database) and tied to logged-in users:

```
@login_required def  
add_to_cart(request, pk):  
    product = get_object_or_404(Product, pk=pk)    cart_item, created =  
Cart.objects.get_or_create(user=request.user, product=product)    if not created:  
        cart_item.quantity += 1  
cart_item.save()    messages.success(request,  
'Added to cart!')    return redirect('shop')
```

```
@login_required def  
cart_view(request):  
    cart_items = Cart.objects.filter(user=request.user)  
total = sum(item.get_total() for item in cart_items)  
return render(request, 'cart.html', {'cart_items':  
cart_items, 'total': total})
```

```
@login_required def  
checkout(request):
```

```

    cart_items = Cart.objects.filter(user=request.user)    total =
sum(item.get_total() for item in cart_items)    order =
Order.objects.create(user=request.user, total_amount=total)

```

```

    for item in cart_items:
OrderItem.objects.create(
order=order,
product=item.product,
quantity=item.quantity,
price=item.product.price
    )
    cart_items.delete()
messages.success(request, 'Order placed!')
return redirect('home')

```

```

7.3 Sales Reports (Accountant Feature) @role_required(['accountant',
'admin']) def sales_report(request):    orders = Order.objects.all()    total_sales =
orders.aggregate(Sum('total_amount'))['total_amount__sum'] or 0    total_orders
= orders.count()    return render(request, 'accountant/sales_report.html', {
    'orders': orders,
    'total_sales': total_sales,
    'total_orders': total_orders
    })

```

STEP 8: URL CONFIGURATION

8.1 Shop URLs (shop/urls.py)

```

from django.urls import path from
. import views

```

```

urlpatterns = [    # Authentication    path('signup/',
views.signup_view, name='signup'),    path('login/',
views.login_view, name='login'),    path('logout/',
views.logout_view, name='logout'),

    # Public pages    path("", views.home_view, name='home'),    path('shop/',
views.shop_view, name='shop'),    path('product/<int:pk>/',
views.product_detail_view, name='product_detail'),

    # Cart    path('cart/', views.cart_view, name='cart'),    path('add-to-cart/<int:pk>/',
views.add_to_cart, name='add_to_cart'),    path('remove-from-cart/<int:pk>/',
views.remove_from_cart, name='remove_from_cart'),    path('checkout/', views.checkout,
name='checkout'),

    # Admin CRUD    path('admin/categories/', views.category_list,
name='category_list'),    path('admin/categories/create/', views.category_create,
name='category_create'),    path('admin/products/', views.product_list,
name='product_list'),    path('admin/products/create/', views.product_create,
name='product_create'),

    # Reports    path('accountant/sales/', views.sales_report,
name='sales_report'),
]

```

8.2 Main URLs (ecommerce/urls.py)

```

from django.contrib import admin from
django.urls import path, include from

```

```
django.conf import settings from
django.conf.urls.static import static
```

```
urlpatterns = [    path('admin/',
admin.site.urls),    path("",
include('shop.urls')),
]
```

```
if settings.DEBUG:    urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)
```

STEP 9: DJANGO ADMIN PANEL

9.1 Registering Models (shop/admin.py) from django.contrib

```
import admin from .models import Role, Category, Product, Cart,
Order, OrderItem
```

```
admin.site.register(Role) admin.site.register(Category)
admin.site.register(Product) admin.site.register(Cart)
admin.site.register(Order)
admin.site.register(OrderItem)
```

9.2 Creating Superuser python

```
manage.py createsuperuser #
Username: admin
# Email: admin@example.com
# Password: admin123
```

The Django admin panel is accessible at <http://127.0.0.1:8000/admin/> and allows:

- Managing users and assigning roles
- Performing CRUD operations on all models

- Viewing orders and cart items

STEP 10: GITHUB VERSION CONTROL

10.1 Initializing Git Repository

```
git init
```

```
add .
```

```
git commit -m "Initial Django project setup with MySQL"
```

10.2 Creating .gitignore

```
*.pyc
```

```
__pycache__/
```

```
media/ *.log
```

```
db.sqlite3
```

10.3 Pushing to GitHub

```
git remote add origin https://github.com/yourusername/ecommerce-lab3.git
```

```
git branch -M main
```

```
git push -u origin main
```

10.4 Regular Commits

Throughout development, I made commits for each major feature:

- `git commit -m "Added authentication system"`
- `git commit -m "Implemented CRUD for categories and products"`
- `git commit -m "Added shopping cart functionality"`
- `git commit -m "Created sales report for accountants"`

SCREENSHOTS SECTION:

1. Admin login page

Django administration

Username:

admin

Password:

.....

Log in

Figure 1 Admin Login Portal

Django administration

Site administration

AUTHENTICATION AND AUTHORIZATION

Groups

+ Add

Change

Users

+ Add

Change

SHOP

Carts

+ Add

Change

Categories

+ Add

Change

Order items

+ Add

Change

Orders

+ Add

Change

Products

+ Add

Change

Roles

+ Add

Change

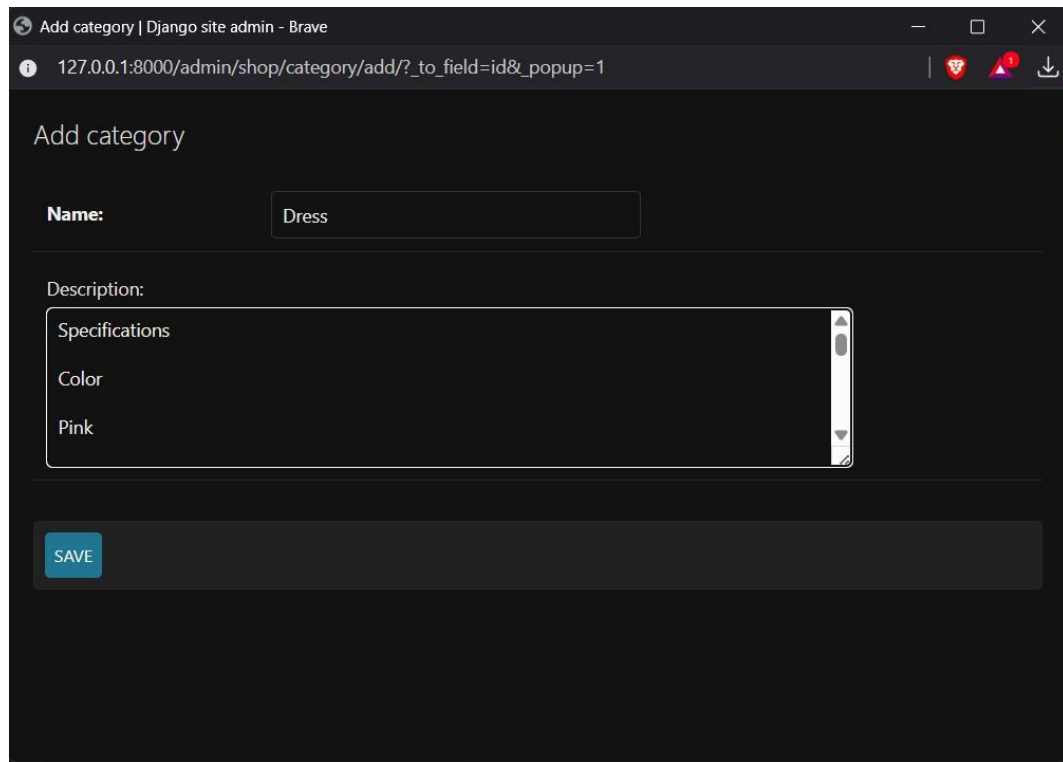
Recent actions

My actions

None available

Figure 2 Admin Dashboard

2. CRUD Operations in Django admin for Products and Categories



The screenshot shows a web browser window titled "Add category | Django site admin - Brave". The address bar displays the URL "127.0.0.1:8000/admin/shop/category/add/?_to_field=id&_popup=1". The page content is titled "Add category". It features a form with two main sections: "Name:" and "Description:". The "Name:" section has a text input field containing the word "Dress". The "Description:" section has a larger text area containing the text "Specifications", "Color", and "Pink" on separate lines. At the bottom of the form is a blue button labeled "SAVE".

Figure 3 Adding Category from Admin Dashboard

Add product

Name: Baby Pink Knitted Two-Piece Set with Straples

Description:

The Pink Knitted Two-Piece Set blends sleek minimalism with a touch of edge. The black strapless top features a twist-front detail and a subtle front slit, adding both structure and allure. Paired with a figure-hugging knitted skirt, this set creates a refined silhouette that's perfect for evening outings or statement day looks.

Made with a premium polyester-lycra blend, it offers gentle stretch for comfort while maintaining its shape. The black strapless top complements the high-waisted fitted skirt, making this co-ord an effortless choice for any occasion where you want to turn heads.

Price: 2200

Stock: 40

Category: Dress

Image: Choose File Screenshot ... 170940.png

SAVE Save and add another Save and continue editing

Figure 4 Adding Products from Admin Dashboard

3. User and Groups setup

Add group

Name: Authenticator

Permissions:

Available permissions

admin | log entry | Can add log entry
admin | log entry | Can change log entry
admin | log entry | Can delete log entry
admin | log entry | Can view log entry
auth | group | Can add group
auth | group | Can change group
auth | group | Can delete group
auth | group | Can view group
auth | permission | Can add permission
auth | permission | Can change permission
auth | permission | Can delete permission
auth | permission | Can view permission

Chosen permissions

Choose all Remove all

Hold down "Control", or "Command" on a Mac, to select more than one.

SAVE Save and add another Save and continue editing

Figure 5 Groups Setup in Admin

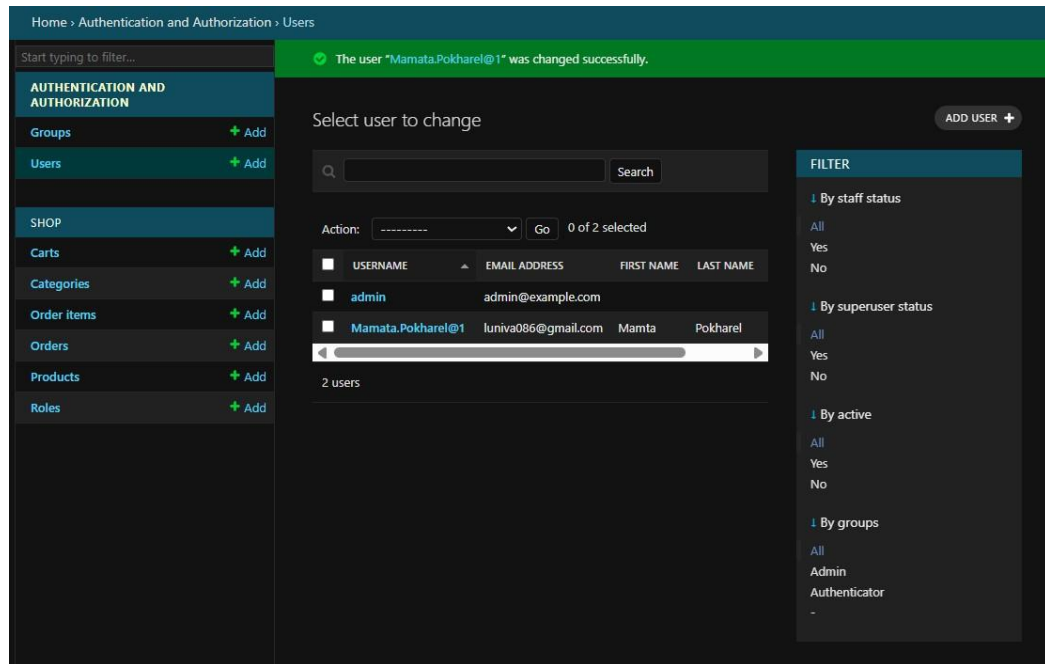


Figure 6 Users Setup in Admin Dashboard

4. Product Page

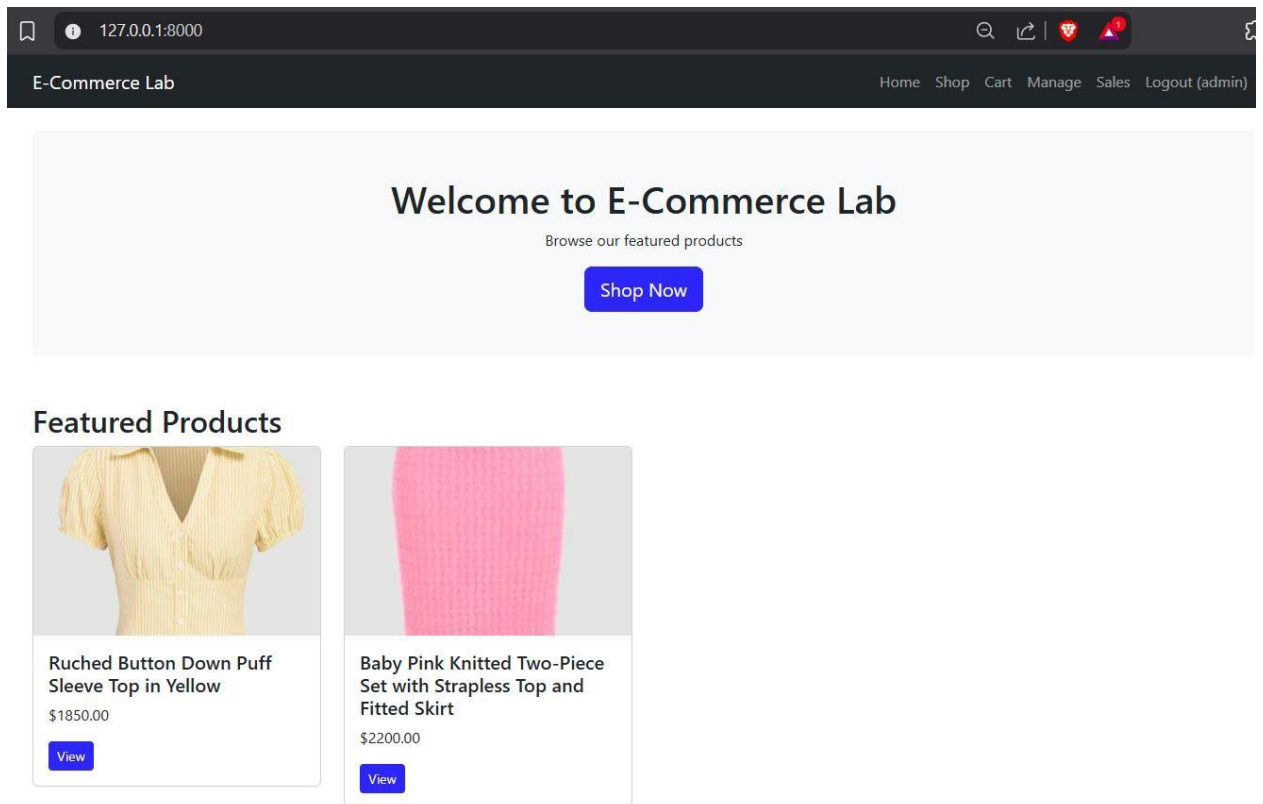


Figure 7 Added Products in Product Page

5. Product Description Page

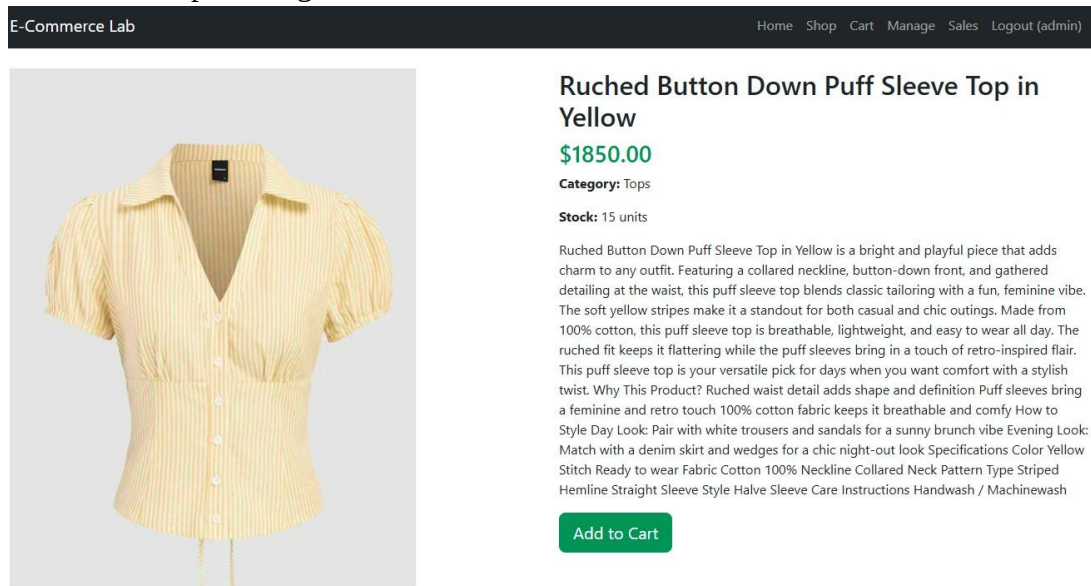


Figure 8 Product Description

6. GitHub repository commits and pushes

```
(venv) PS C:\projects\ecommerce_lab3> git add .
(venv) PS C:\projects\ecommerce_lab3> git commit -m "Setup complete: admin, models, and database connection"
On branch feature/cart
Your branch is up to date with 'origin/feature/cart'.

nothing to commit, working tree clean
(venv) PS C:\projects\ecommerce_lab3> git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Samragyee0X/ecommerce_lab3.git
 5132604..38ed40e  main -> main
(venv) PS C:\projects\ecommerce_lab3> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...
```

Figure 9 GitHub Commit and Push

CONCLUSION

This project successfully demonstrated the complete development lifecycle of a Django web application. I built a functional mini e-commerce platform with authentication, role-based access control, CRUD operations, shopping cart, order management, and sales reporting.

The project reinforced my understanding of backend web development concepts including database design, ORM usage, user authentication, file uploads, and version control. The hands-on experience with Django's built-in features (admin panel, authentication system, template engine) highlighted why Django is considered a "batteries-included" framework.

Future enhancements could include:

- Payment gateway integration
- Email notifications for orders
- Advanced search and filtering
- Product reviews and ratings
- Inventory management system
- RESTful API for mobile app integration

REFERENCES

- Django Documentation: <https://docs.djangoproject.com/>
- MySQL Documentation: <https://dev.mysql.com/doc/>
- Git Documentation: <https://git-scm.com/doc>
- Bootstrap 5 Documentation: <https://getbootstrap.com/docs/5.3/>